Designing Workshops to Bridge Visual and Textual Programming: A Narrative Game Approach

André Matutat
Benjamin Franke
Niklas Schumann
Birgit Christina George
Jörg-Michael Keuntje
Carsten Gips
andre.matutat@hsbi.de
benjamin.franke@hsbi.de
niklas.schumann@hsbi.de
bc.george@hsbi.de
joerg-michael.keuntje@hsbi.de
carsten.gips@hsbi.de
Hochschule Bielefeld
Bielefeld, Germany

ABSTRACT

This work-in-progress paper addresses the transition gap between visual, text-based, and complex programming skills in early computer science education. We present a multi-level workshop concept designed to provide a coherent and motivating learning progression across different programming stages. The workshops integrate narrative-driven gameplay with progressively challenging programming tasks in a custom-developed 2D dungeon crawler framework, aiming to support learners in progressing from block-based coding to text-based and open code manipulation. The concept was implemented in collaboration with regional schools and industry partners and evaluated in six workshops with 92 participants. Initial results from pre- and post-surveys indicate high acceptance and a positive trend in programming interest, particularly among female students.

CCS CONCEPTS

• **Applied computing** → **Interactive learning environments**; *Collaborative learning*.

KEYWORDS

Block-based programming, Text-based programming, Game-based learning, Narrative-centered learning, Programming workshops, Transition to text-based coding,

ACM Reference Format:
André Matutat, Benjamin Franke, Niklas Schumann, Birgit Christina George, Jörg-Michael Keuntje, and Carsten Gips. 2026. Designing Workshops to Bridge Visual and Textual Programming: A Narrative Game Approach. In *10th Annual International GamiFIN Conference 2026 (GamiFIN 2026), March 23–27, 2026, Saariselkä, Finland*. ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3789624.3789625

Please use nonacm option or ACM Engage class to enable CC licenses
This work is licensed under a Creative Commons Attribution 4.0 International License.
GamiFIN 2026, March 23–27, 2026, Saariselkä, Finland
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2190-8/2026/03
https://doi.org/10.1145/3789624.3789625

1 INTRODUCTION

Germany faces a persistent shortage of skilled professionals in computer science and related STEM disciplines. Emerging technologies, such as artificial intelligence, have increased the demand for well-trained specialists [8]. At the same time, schools and universities struggle to provide students with sufficient exposure to programming and digital literacy, while women remain underrepresented in IT programs and professions [5, 9]. Early and engaging interventions are therefore essential to foster interest in computer science and to build long-term motivation [2].

Block-based programming environments such as Scratch and Blockly have become popular tools for introducing young learners to the fundamentals of programming and computational thinking. Scratch enables children to create games through a drag-and-drop interface that eliminates syntax errors and fosters creativity [14]. Similarly, Google's Blockly provides a visual block interface that can be mapped to text-based code (e.g., JavaScript), making it suitable as a bridge toward more advanced programming environments.

The literature identifies several advantages of block-based programming. Such environments lower the barrier to entry by reducing syntax-related difficulties and enabling early engagement with programming concepts [17]. Prior experience with visual programming can support the transition to text-based languages [3]. In addition, immediate visual feedback and open-ended design opportunities promote motivation, creativity, and early computational thinking, which can later be transferred to text-based coding [6, 11].

In educational practice, block-based programming is widely used in both formal and informal learning contexts, including school curricula and after-school clubs. Despite their pedagogical success, two persistent challenges remain. First, the transition from block-based to text-based or more advanced programming environments is often insufficiently supported, leading to conceptual gaps at

higher levels of abstraction [1]. While approaches such as Droplet [4] or PyBlox [10] aim to address this transition, they primarily focus on introductory text-based programming and provide limited support for more open-ended or advanced contexts. Second, many block-based learning activities remain decontextualized and rely on isolated exercises rather than coherent, story-driven challenges. Prior research shows that narrative and storytelling approaches can significantly enhance engagement and learning outcomes [13, 15, 16].

Building on these insights, the present work introduces a narrative-driven workshop concept designed to support learners across multiple programming skill levels. The concept goes beyond introductory block-to-text transitions by providing a coherent progression from visual to more advanced text-based programming, embedded in a shared game-based environment that leverages storytelling to enhance motivation and contextualize learning across stages.

Funded through a regional initiative, the project offers programming workshops for students in grades 5–13, developed in collaboration with regional schools and industry partners. Using a self-developed 2D dungeon crawler, the workshops embed programming tasks in a story-driven game world to spark early interest in STEM, particularly among pupils new to computer science. Designed as low-threshold entry points, the workshops support recurring participation across increasing levels of difficulty.

This paper presents the first phase of an ongoing research project, with results representing work in progress that will be expanded and validated in future workshop iterations.

2 METHODOLOGY

Three distinct skill groups within the target audience were identified, each corresponding to a dedicated workshop format. Pupils aged 10–12 with no prior programming experience participate in the visual *Blockly-Dungeon*, which introduces programming through block-based interaction and basic algorithmic thinking. Learners aged 11–14 with prior experience in visual programming environments such as Blockly or Scratch attend the *Java-Dungeon*, where they transition to text-based programming and apply core constructs such as loops, conditionals, and functions. Pupils aged 15–17 with prior knowledge of text-based languages such as Java or Python take part in the *Advanced-Dungeon*, which focuses on more complex programming concepts and open-ended problem-solving tasks. These skill levels and workshop formats were defined collaboratively with partner schools to align with students' prior experience and curricular expectations.

Each skill level addresses specific learning objectives that build upon one another to form a coherent progression. In the *Blockly-Dungeon*, participants develop fundamental programming skills and basic algorithmic thinking. The *Java-Dungeon* encourages learners to write their first text-based programs and apply core programming constructs in meaningful contexts. The *Advanced-Dungeon* enables students to extend prior knowledge, explore more complex concepts such as arrays and sorting algorithms, and engage in advanced problem-solving within a gamified, narrative-driven environment. Collectively, this multi-level structure supports technical skill development while leveraging the motivational effects of story-driven, game-based learning.

A central concept of the project is to enable pupils to progress through the workshops sequentially. Participants may begin with the *Blockly-Dungeon* and later return for the *Java-Dungeon* and *Advanced-Dungeon*, creating a continuous narrative learning experience and educational connections between workshops. As programming is also taught within the regular school curriculum, the workshops serve as supplementary, experience-oriented learning events. Consequently, longer intervals-typically one semester-occur between participation in successive workshops.

2.1 Dungeon

Building on prior work on block-based and text-based programming environments and the need for coherent progression and narrative framing, this section presents the conceptual design of a multi-level, gamified programming learning environment.

All activities are embedded within a custom-developed 2D dungeon crawler environment, referred to as the *Dungeon*[1]. The framework enables the integration of programming tasks into interactive game scenarios through dedicated levels and challenges for each workshop type and skill level, while consistently reusing the same game world, characters, and core mechanics to create narrative and structural continuity across workshop levels.

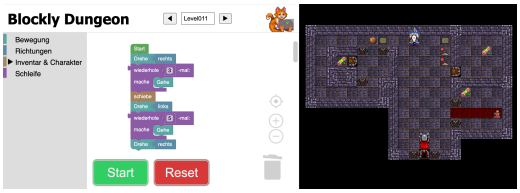


Figure 1: The complete Blockly-Dungeon environment. The interface combines the Blockly workspace (left) with the corresponding 2D *Dungeon* level (right), linking visual programming directly to in-game actions and puzzles.

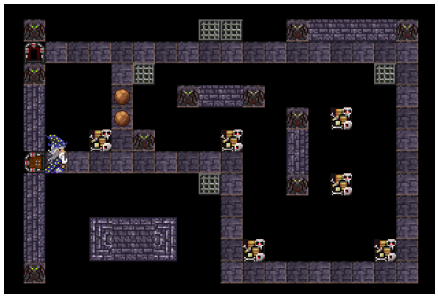
Figure 1 shows the complete *Blockly-Dungeon* environment, combining the visual Blockly interface with the interactive *Dungeon* game. On the left, participants assemble code blocks to control their character; on the right, the resulting actions are executed in the game, where players solve puzzles to progress through the *Dungeon*.

Each *Dungeon* variant is implemented through a dedicated interface aligned with its respective skill level. The *Blockly-Dungeon* uses a web-based implementation built on Google's Blockly library, allowing participants to assemble visual code blocks to control game elements. The *Java-Dungeon* connects to the game via a custom Visual Studio Code extension, enabling learners to write and execute Java code that directly influences gameplay. Both versions share the same levels and allow dynamic switching between visual and textual input modes, supporting a smoother transition between the two paradigms. Unlike approaches such as those described by Alrubaye et al. [1], where textual code is merely generated from visual blocks, participants in the *Java-Dungeon* are required to write their own code, actively engaging in the transition from visual

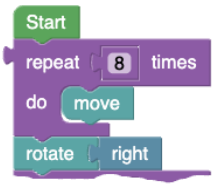
[1]The Dungeon Framework is available as open-source software on GitHub: https://github.com/Dungeon-CampusMinden/Dungeon

to text-based programming. This approach aligns with findings by Weintrop et al. [17], who emphasize the pedagogical value of environments that allow learners to work across both modalities and to develop a deeper understanding of the relationship between block-based and textual representations.

Participants in the *Advanced-Dungeon* can freely explore the level, discover riddles and hints embedded in the environment, and locate related sections in the game's source code. To progress, they must identify and fix bugs directly in the code. Through hot reloading, changes take effect instantly, allowing immediate feedback and experimentation. Together, these implementations create a seamless transition from block-based to text-based programming and ultimately to open code manipulation, strengthening both conceptual understanding and engagement through tangible, in-game results.



(a) A dungeon level. The player starts at the bottom left and must follow a narrow path with a chasm to reach the exit at the upper left.



(b) The corresponding Blockly code, combining loops and movement/rotation commands to solve the level.

```
for(int i = 1; i < 9; i++)
    hero.move();
hero.rotate(Direction.RIGHT);
for(int i = 1; i < 6; i++)
    hero.move();
hero.rotate(Direction.LEFT);
for(int i = 1; i < 9; i++)
    hero.move();
hero.rotate(Direction.LEFT);
for(int i = 1; i < 11; i++)
    hero.move();
hero.rotate(Direction.LEFT);
for(int i = 1; i < 18; i++)
    hero.move();
```

(c) Equivalent Java solution for the same level.

Figure 2: The Blockly-Dungeon setup for beginner workshops. (a) shows the game environment, (b) the Blockly solution, and (c) the equivalent Java implementation.

Figure 2 illustrates the progression from block-based to text-based programming within the *Dungeon* environment. The example shows a dungeon level solved using Blockly blocks and the corresponding Java implementation, highlighting the correspondence between visual and textual representations of loops, iteration, and control flow.

2.2 Workshops

Each workshop took place over two afternoons, with about four hours of interactive activity per day. Sessions took place on campus using preinstalled PCs or Raspberry Pi workstations running the respective *Dungeon* environments. Depending on the workshop type, participants used the browser-based *Blockly-Dungeon*, a Visual Studio Code extension for the *Java-Dungeon*, or IntelliJ for the *Advanced-Dungeon*.

Participants were pupils who joined the workshops voluntarily, forming mixed groups of approximately 10–20 learners supported by two to four instructors. Suggested workshop types and age or experience levels served as guidelines during registration but were not strictly enforced, allowing participants to choose according to their interests. While each participant had their own workstation, collaboration and pair programming were promoted, given their effectiveness in supporting programming education for novices [7, 12]. Progress was largely self-directed, with instructors providing guidance when needed, and sessions focused on hands-on programming activities within the *Dungeon* environment. Short industry contributions connected workshop content to real-world programming practice.

3 RESULTS

This section presents work-in-progress results from the initial phase of the workshop series. The results are descriptive and provide an initial overview of participation patterns and self-reported programming interest. No inferential statistical analyses were conducted at this stage, and further data collection is ongoing.

Participants completed anonymous pre- and post-surveys assessing self-reported programming interest and engagement using identical items on a 5-point Likert scale (1 = not interested, 5 = very interested).

Attendance data and basic demographic information (age, gender, and prior programming experience) were collected to contextualize the findings.

The collected data provide an initial overview of participation, engagement, and interest development across workshop types. As shown in Table 1, retention was generally high, particularly in the *Java-* and *Advanced-Dungeon* workshops. Table 2 presents programming interest by gender and workshop type, showing a positive trend across most groups, with the strongest increase observed among female participants. Similarly, Table 3 presents programming interest levels by age group, revealing distinct trends across workshops. In the *Blockly-Dungeon* workshop, interest slightly decreased among participants aged 10–12 and those 19 and older, while it increased among the 16–18 age group. In the *Java-Dungeon* workshop, interest generally rose, except for a minor decline among the 13–15-year-olds. In the *Advanced-Dungeon* workshop, interest

Table 1: Participant numbers and average self-reported programming interest before and after the workshops. Subtable (a) reports the number of participants who completed the pre- and post-surveys as well as the retention rate between them. Subtable (b) presents the corresponding mean interest ratings on a 5-point Likert scale (1 = not interested, 5 = very interested).

(a) Participants per workshop and survey phase.

Workshop	Pre	Post	Retention (%)
Blockly	41	29	70.7
Java	32	31	96.9
Advanced	19	18	94.7
Total	**92**	**78**	**84.8**

(b) Average self-reported interest in programming (1–5) grouped by the workshop type.

Workshop	M (pre)	M (post)	Md (pre)	Md (post)
Blockly	3.76	3.83	4	4
Java	4.25	4.39	4	4
Advanced	4.63	4.89	5	5
Overall mean	**4.12**	**4.29**	4.00	4.00

Table 2: Average self-reported interest in programming before and after the workshops, grouped by gender and workshop type (1 = not interested, 5 = very interested). *n* refers to the number of participants per post-survey.

Blockly

Gender	M (pre)	M (post)	Md (pre)	Md (post)	*n*
Female	3.56	4.17	3	4	6
Male	3.81	3.74	4	4	23
Non-Binary	–	–	–	–	0

Java

Gender	M (pre)	M (post)	Md (pre)	Md (post)	*n*
Female	3.89	4.11	4	4	9
Male	4.39	4.50	4	4.5	22
Non-Binary	–	–	–	–	0

Advanced

Gender	M (pre)	M (post)	Md (pre)	Md (post)	*n*
Female	4.50	5.00	5	5	3
Male	4.71	4.86	5	5	14
Non-Binary	4.00	5.00	4	5	1

among the 13–15-year-olds remained stable at a very high level, and the 16–18 group showed a low increase.

Table 4 summarizes programming interest by prior experience and workshop type. Notably, participants without prior experience exhibited a decrease in interest in the *Blockly-Dungeon* workshop, while interest among more experienced participants remained high or slightly increased across workshops.

Table 3: Average self-reported interest in programming before and after the workshops, grouped by age and workshop type (1 = not interested, 5 = very interested).

Blockly

Age Group	M (pre)	M (post)	Md (pre)	Md (post)
10–12	4.27	4.22	5.00	4.00
13–15	–	–	–	–
16–18	2.93	3.20	3.00	3.50
19 and older	3.80	3.00	3.50	3.00

Java

Age Group	M (pre)	M (post)	Md (pre)	Md (post)
10–12	4.00	4.27	4.00	4.00
13–15	4.50	4.43	4.50	4.00
16–18	4.38	4.50	4.00	4.50
19 and older	–	–	–	–

Advanced

Age Group	M (pre)	M (post)	Md (pre)	Md (post)
10–12	–	–	–	–
13–15	5.00	5.00	5.00	5.00
16–18	4.59	4.88	5.00	5.00
19 and older	–	–	–	–

Overall, the results suggest that the workshop structure largely maintained or slightly increased motivation across most learner groups. However, the decline in interest among participants without prior experience in the introductory *Blockly-Dungeon* indicates that

Table 4: Average self-reported interest in programming by prior programming experience (visual or text based) and workshop type (1 = not interested, 5 = very interested).

Blockly

Experience Level	M (pre)	M (post)	Md (pre)	Md (post)
No prior experience	2.87	2.43	3.00	2.00
Some experience	4.27	4.27	4.00	4.00

Java

Experience Level	M (pre)	M (post)	Md (pre)	Md (post)
No prior experience	3.50	4.00	3.50	4.00
Some experience	4.30	4.41	4.00	4.00

Advanced

Experience Level	M (pre)	M (post)	Md (pre)	Md (post)
No prior experience	3.00	–	3.00	–
Some experience	4.72	4.89	5.00	5.00

further design adjustments may be required. These findings are preliminary and reflect only the first phase of the ongoing study.

4 DISCUSSION

The initial evaluation indicates that the workshop concept is feasible and well received across all three skill levels. Participants generally reported a high interest in programming, with a slight overall increase after participation. However, since baseline interest was already high, the observed change must be interpreted with caution.

Furthermore, the survey data are based on self-reported measures and cannot be directly linked to individual participants across both survey phases. Students with lower motivation were more likely to discontinue participation and therefore not complete the post-survey. As a result, the post-workshop data may overrepresent more motivated learners. This selective retention can artificially inflate observed changes in programming interest, as apparent increases may reflect differential dropout rather than genuine changes in individual motivation. Due to the anonymous data collection and the lack of participant-level matching between pre- and post-surveys, this effect cannot be disentangled or corrected within the current dataset. In addition, subgroup sizes were relatively small, particularly when divided by age or prior experience, further limiting the robustness of the observed trends.

At the same time, the current evaluation does not include objective measures of learning outcomes or conceptual understanding. While the workshop concept is designed to support transitions between visual, text-based, and more advanced programming, this first project phase does not provide direct evidence of learning gains or transfer between programming modalities.

Some decreases in reported interest, particularly among beginners in the *Blockly-Dungeon*, may be explained by contextual factors ratheTgr than the workshop concept itself. Early workshop iterations included technical issues and software bugs that occasionally disrupted the flow of activities, and participant feedback indicated that industry contributions were less engaging for younger audiences. Since these early iterations, substantial technical refinements have been implemented, resulting in improved software stability, and the role and format of industry contributions have been critically reviewed and adapted based on participant feedback.

At this stage, it is not yet possible to assess how effectively the workshops bridge the conceptual gap between block-based, text-based, and more complex programming. Only the first phase of the multi-level program has been completed, and longitudinal observations and objective assessments are required to examine whether participants progress across levels as intended.

Overall, the findings align with previous research highlighting the motivational potential of game-based and narrative programming environments [16, 17].

5 FUTURE WORK

Future work will focus on extending data collection across subsequent workshop phases and exploring more objective approaches to evaluating learning processes and transitions between programming modalities. Future evaluations will move beyond self-reported measures to better capture conceptual understanding and progression over time, supported by additional workshop iterations and a broader empirical basis. Based on participant feedback, industry contributions will be omitted for younger target groups, while talks for older participants will be redesigned to be more interactive. Future work will further focus on analyzing learner trajectories across the different workshop levels over time, with particular emphasis on transitions between visual, text-based, and more open programming modalities. Finally, suitable evaluation approaches for capturing learning processes and modality transitions in multi-level, workshop-based programming settings will be further developed and empirically explored.

ACKNOWLEDGMENTS

This project was carried out at Bielefeld University of Applied Sciences and funded by the European Union and the State of North Rhine-Westphalia as part of the EFRE/JTF funding program "Pakt für Informatik 2.0" (Funding Code: EFRE-20300105).

Generative AI tools were used to support language editing and improve clarity and conciseness of the manuscript; all content, interpretations, and conclusions remain the sole responsibility of the authors.

REFERENCES

[1] Hussein Alrubaye, Stephanie Ludi, and Mohamed Wiem Mkaouer. 2019. Comparison of block-based and hybrid-based environments in transferring programming skills to text-based environments. arXiv:1906.03060 [cs.CY] https://arxiv.org/abs/1906.03060

[2] Laura Rodr
íguez Amaya, Mina Guirguis, and Jocabed Marquez. 2019. Redesigning Computer Science: An Early Intervention to Combat Undergraduate Student Misconceptions. *Research Report for NASA Educator Professional Development Collaborative* 1, 2 (06 2019), 1–6. https://docs.gato.txst.edu/176251/Redesigning%20Computer%20Science%20Perceptions_RR_06042019.pdf

[3] Michal Armoni, Orni Meerbaum-Salant, and Mordechai Ben-Ari. 2015. From scratch to "real" programming. *ACM Transactions on Computing Education (TOCE)* 14, 4 (2015), 1–15.

[4] David Bau. 2015. Droplet, a blocks-based editor for text code. *Journal of Computing Sciences in Colleges* 30, 6 (2015), 138–144.

[5] Bitkom e.V. 2025. *Förderung von Frauen in IT- und Digitalberufen 2025.* Technical Report. Bitkom e.V. https://doi.org/10.64022/2025-frauen-it-digitalberufe

[6] Shuchi Grover and Roy Pea. 2013. Computational thinking in K–12: A review of the state of the field. *Educational researcher* 42, 1 (2013), 38–43.

[7] Brian Hanks, Sue Fitzgerald, Renée McCauley, Laurie Murphy, and Carol Zander. 2011. Pair programming in education: A literature review. *Computer Science Education* 21, 2 (2011), 135–173.

[8] Institut der deutschen Wirtschaft Köln. 2024. MINT-Herbstreport 2024 – Arbeitsmarktbericht. https://www.nationalesmintforum.de/fileadmin/medienablage/user_upload/MINT-Herbstreport_2024_Arbeitsmarktbericht.pdf

[9] IU Internationale Hochschule. 2022. Digitale Bildung in Deutschland. https://static.iu.de/studies/Digitale_Bildung_Studie.pdf

[10] Devin Jean, Gordon Stein, Brian Broll, and Ákos Lédeczi. 2025. Easing the Block-to-Text Transition: A Scaffolded Approach to Learning Python. In *Proceedings of the ACM Global on Computing Education Conference 2025 Vol 1* (Gaborone, Botswana) *(CompEd 2025)*. Association for Computing Machinery, New York, NY, USA, 113–119. https://doi.org/10.1145/3736181.3747126

[11] Filiz Kalelioglu and Yasemin Gülbahar. 2014. The Effects of Teaching Programming via Scratch on Problem Solving Skills: A Discussion from Learners' Perspective. *Informatics in education* 13, 1 (2014), 33–50.

[12] Phil Maguire, Rebecca Maguire, Philip Hyland, and Patrick Marshall. 2014. Enhancing collaborative learning using pair programming: Who benefits? *AISHE-J: The All Ireland Journal of Teaching and Learning in Higher Education* 6, 2 (2014), 1411–1425.

[13] Wookhee Min, Bradford Mott, Kyungjin Park, Sandra Taylor, Bita Akram, Eric Wiebe, Kristy Elizabeth Boyer, and James Lester. 2020. Promoting Computer Science Learning with Block-Based Programming and Narrative-Centered Gameplay. In *2020 IEEE Conference on Games (CoG)*. IEEE, Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 654–657. https://doi.org/10.1109/CoG47356.2020.9231881

[14] Mitchel Resnick, John Maloney, Andrés Monroy-Hernández, Natalie Rusk, Evelyn Eastmond, Karen Brennan, Amon Millner, Eric Rosenbaum, Jay Silver, Brian Silverman, and Yasmin Kafai. 2009. Scratch: programming for all. *Commun. ACM* 52, 11 (Nov. 2009), 60–67. https://doi.org/10.1145/1592761.1592779

[15] Jonathan P Rowe, Scott W McQuiggan, Bradford W Mott, and James C Lester. 2007. Motivation in narrative-centered learning environments. In *Proceedings of the workshop on narrative learning environments, AIED*. 40–49.

[16] Andy Smith, Danielle Boulden, Bradford Mott, Aleata Hubbard-Cheuoua, James Minogue, Kevin Oliver, and Cathy Ringstaff. 2021. Supporting Interactive Storytelling with Block-Based Narrative Programming. In *Interactive Storytelling*, Alex Mitchell and Mirjam Vosmeer (Eds.). Springer International Publishing, Cham, 416–424.

[17] David Weintrop. 2019. Block-based programming in computer science education. *Commun. ACM* 62, 8 (July 2019), 22–25. https://doi.org/10.1145/3341221